10. Typescript & Testing

Attention!

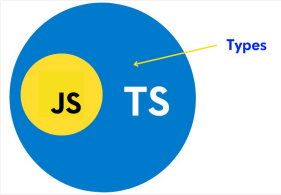
This training is **interview-driven**.

To become a qualified developer, *ChatGPT* and *YouTube* will always be your best friends.

Outline

- Typescript
- Testing

Typescript



- TypeScript (TS): JavaScript + Types
  - Cannot be directly executed by the browser, must be compiled to JS
- Why TypeScript?
  - Build complex applications in large teams
  - Catch simple errors before runtime, type safety
  - Improved readability, quality, scalability and maintainability
  - Better tooling, Intellisense

Primitive types

- Always use **lowercase** primitive types

Type	Example
`string`	`let name: string = "Alice";`
`number`	`let age: number = 30;`
`boolean`	`let isAdmin: boolean = true;`
`null`	`let nothing: null = null;`
`undefined`	`let notAssigned: undefined = undefined;`
`bigint`	`let big: bigint = 100n;`
`symbol`	`let sym: symbol = Symbol("id");`

Types

- **Array:** List of values of same type
  - type[] or Array<type>
- **Tuple**: Fixed-size array with mixed types
  - Representing data with a **fixed structure**
  - maintain the order and type
- **enum:** Named set of constant values
  - variable should have a **set of predefined values**
- **Literal Types**
  - Exact value or small union of values
    - Create strict value sets

Types

- **Any:** Turn off TypeScript, disables type checking
  - The type could be anything
- **Unknown**
  - Safer “any”
  - Can assign anything to unknown, forces **type checking** before use
- **void:** No meaningful return value
  - The function does not return a value
    - Usage: A function has side effects but doesn’t return anything meaningful
- **Never:** A type for functions that never return
  - Mark functions that don’t complete normally
    - throws an error or has an infinite loop

any vs unknown vs void vs never

Type	Can hold values?	Can be used directly?	Purpose
`any`	Yes (anything)	Yes (dangerous)	Disable type checking
`unknown`	Yes (anything)	No (must narrow)	Safe untyped input
`void`	Only `undefined`	No	No meaningful return
`never`	No values	No	Impossible code path

Interfaces vs type

- Type Alias (type)
  - More flexible than interfaces
    - Great for type combinations
  - Extend: **&**
- interface ({})
  - mainly used to describe object shapes
  - Extend: **extends**

Interfaces vs type

Feature	`interface`	`type`
Object structure	✅ Yes	✅ Yes
Class implementation	✅ Preferred	✅ Allowed
Extend / inheritance	✅ With `extends`	✅ With `&`
Declaration merging	✅ Yes	❌ No
Union / literal / tuple	❌ No	✅ Yes
Function signature	✅ Yes (less common)	✅ Yes (more common)
React props	✅ Common	✅ Common

Type Assertion (as)

- Type assertion tells the TS compiler (TSC) to treat a variable as a different type
  - manually tell the compiler what type a value is
  - **as** Syntax (Recommended)
    - In React projects, always use as syntax
  - Chained assertions

TypeScript operator

- Value-level operators
  - typeof
- Type-level operators
  - Union operator |: either
  - Intersection operator &: both
  - keyof: Extracts keys of an object type
- Conditional operator
  - Optional chaining (**?.**)
    - If the value is undefined or null, it short-circuits and returns undefined
    - Prevents runtime errors
    - Working with deeply nested objects or optional values

Generics (<>)

- Generics let a function work with different types while keeping the exact type it was called with, instead of losing it to any or unions
  - placeholders for types, let you define type variables that are specified later
  - Avoid code duplication for different types
  - Ensure type safety — no “any”
  - Make code more reusable and scalable
- Types:
  - <T>: Declare a generic type parameter T
  - <T, U>: Multiple generic type parameters
  - <T extends X>: Constrain T to types that extend X

Index Signatures vs Record

- Index Signatures
  - Describes objects with dynamic keys
- Record<K, V>
  - An object type whose keys are K and whose values are V
    - If you can list the keys, use Record

❌ Using index signatures when keys are known:

```ts
// BAD
type Roles = {
  [key: string]: boolean;
};
```

✅ Correct:

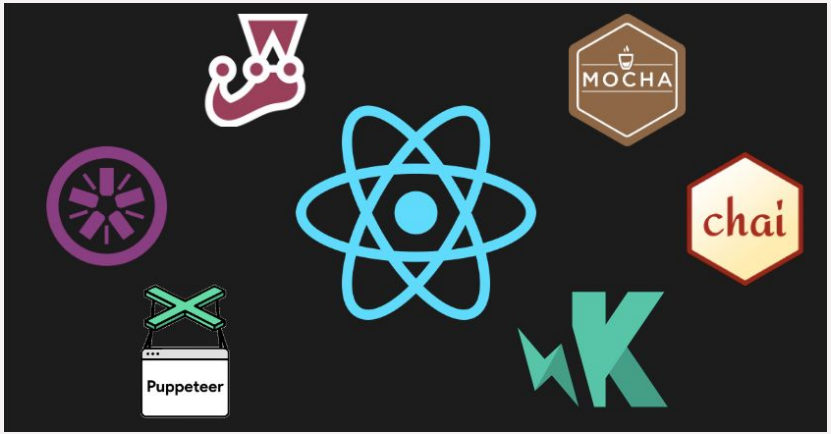
```ts
type Roles = Record<"admin" | "user", boolean>;
```

readonly

- This property can be read, but cannot be reassigned after initialization

Testing

- Testing in React refers to the process of verifying that your React components and application behave as expected
  - Find logical errors or missing requirements
  - Reliability, security, performance



BDD vs TDD

- BDD (Behavior-Driven Development)
  - Describing the behavior of an application from the user's perspective
    - Write tests very similar to spoken language
    - Write software to implement features and test
- TDD (Test-Driven Development)
  - Write a failing test → write minimal code to pass → refactor
  - Write tests before writing the actual code
    - Repeatedly write and test software to pass all test cases

Test Types - Unit test

- **Unit test**
  - Tests a single function or component in isolation
    - React Testing Library (RTL), @testing-library/jest-dom, Jest

ts

```ts
// math.ts
export function add(a: number, b: number) {
  return a + b;
}
```

ts

```ts
// math.test.ts
test("adds two numbers", () => {
  expect(add(1, 2)).toBe(3);
});
```

Test Types

- Integration test
  - Tests multiple components working together
    - React Testing Library, Jest, Enzyme
- Regression test
  - Ensures a fixed bug doesn’t come back
    - Usually written after a bug is found and fixed
- End-to-End (E2E)
  - Test the entire app from the user's perspective
    - Cypress, Selenium

Testing frameworks

- A testing framework provides the tools to:
  - define tests (test, describe, it)
  - run tests
  - make assertions (expect)
  - report results

Code Coverage

- Metric that tells the percentage of code that is run by automated tests
  - High coverage ≠ high quality, but low coverage usually means risky code
  - Good for checking how well unit and integration tests are doing
- npx **jest —coverage**
- Good: 85% to 95%

```
> jest --coverage

 PASS  ./sayHello.test.js
  ✓ returns a personalized greeting (3 ms)

-------------|---------|----------|---------|---------|-------------------
File         | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------|---------|----------|---------|---------|-------------------
All files    |     100 |      100 |     100 |     100 |
 sayHello.js |     100 |      100 |     100 |     100 |
-------------|---------|----------|---------|---------|-------------------
Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        1.554 s
Ran all test suites.
```

Additional materials

- Utility Types

Homework

- Code along with **ONE** of the following videos:
  - https://www.youtube.com/watch?v=iaAdWmAu0TE&t=107s
  - https://www.youtube.com/watch?v=F9gB5b4jgOI
  - https://www.youtube.com/watch?v=50NN3d-Ne1U
  - https://www.youtube.com/watch?v=VAKDr1Isix0
- Take a screen recording of your project, and post it in the Wechat group
- DDL: 01/06/2026

Homework - Short Answer Questions

1. Explain when you should and should not use TypeScript
2. What is static type checking? How can developers benefit from it?
3. What is type inference?
4. How does the array type differ from tuples?
5. What are enums?
6. Explain the difference between type any and unknown.
7. How do you type functions?
8. What is the difference between union and intersection types?
9. What are type guards? What is narrowing?
10. What is type assertion?

Homework - Short Answer Questions

11. Explain the difference type alias vs interface. When should you use each one?
12. What is type readonly?
13. Explain the difference between public, private, protected.
14. What are generics?
15. What is the difference between static type checking (via Flow or Typescript) and checking provided by “PropTypes”?
16. What is TypeScript, and how does it differ from JavaScript?
17. When do you use a return type of never, and how does it differ from void?
18. What are the different kinds of tests?
19. What is the difference between BDD and TDD?

Homework - Short Answer Questions

20. What libraries can we use to test JS? Explain some of their features.
21. How do we test asynchronous code?
22. What should we pay attention to when testing our code?
23. What is code coverage about?